



nextwork.org

VPC Peering



yakubu abdullahi yusuf

A VPC peering connection `pcx-04da84596e4ff0131` / VPC 1 <> VPC 2 has been requested.

`pcx-04da84596e4ff0131` / VPC 1 <> VPC 2

Actions

Pending acceptance

You can accept or reject this peering connection request using the 'Actions' menu. You have until Sunday, 18 January 2026 at 04:20:40 GMT+3 to accept or reject the request, otherwise it expires.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that lets you create your own private network inside AWS where you can run cloud resources like EC2 instances. It is useful because it gives you full control over networking and security

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to configure inbound and outbound rules in security groups to ensure reliable connectivity between instances of different VPC's.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how many different networking and security components such as subnets, route tables, security groups, network ACLs, and gateways have to work together just to enable basic connectivity and a single misconfiguration can affect connectivity.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

This project took me...

This project took me 30 minutes to complete.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

In the first part of my project...

Step 1 - Set up my VPC

In this step, I will create two VPCs from scratch using the visual VPC resource map.

Step 2 - Create a Peering Connection

In this step, I will setup a connection link between the VPCs i.e VPC peering connections.

Step 3 - Update Route Tables

In this step, I will be setting up a way for traffic coming from VPC1 to get to VPC 2 and setting up a way for traffic coming from VPC2 to VPC 1.

Step 4 - Launch EC2 Instances

In this step, I will launch EC2 instances in each of the VPCs.



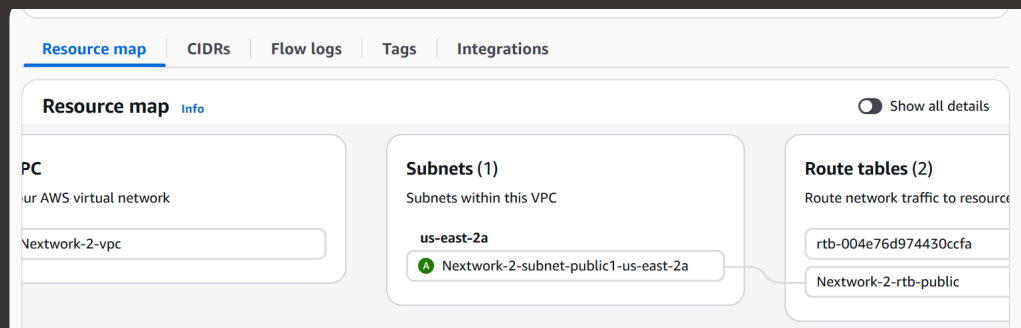
Multi-VPC Architecture

I started my project by launching two VPCs which createdd two subnets.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16 reespectively. They have to be unique to avoid overlap between the resoures from both VPCs.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances because we know a more secured way to connect to our instances and we don't need it in this project.





VPC Peering

A VPC peering connection is a networking connection that allows two VPCs to communicate with each other privately using private IP addresses.

VPCs would use peering connections to communicate with each other.

The difference between a Requester and an Acceptor in a VPC peering connection is their role in initiating and approving the connection.

VPC > Peering connections > Create peering connection

VPC ID (Requester)
vpc-02ad47abe9fba810c (Nextwork-1-vpc)

VPC CIDRs for vpc-02ad47abe9fba810c (Nextwork-1-vpc)

CIDR	Status	Status reason
10.1.0.0/16	Associated	-

Select another VPC to peer with

Account
☒ My account
☐ Another account

Region
☒ This Region (us-east-2)
☐ Another Region

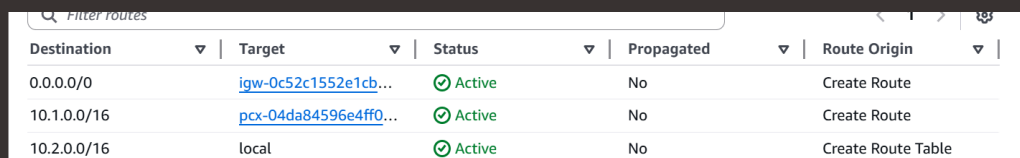
VPC ID (Acceptor)
Select a VPC



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because AWS does not automatically know where to send traffic between the two VPCs.

My VPCs' new routes have a destination of 10.1.0.0/16 and 10.2.0.0/16 The routes' target was VPC 1 <> VPC 2.



The screenshot shows the AWS Management Console interface for route tables. At the top, there is a search bar labeled 'Filter routes'. Below it is a table with the following columns: Destination, Target, Status, Propagated, and Route Origin. The table contains three rows of data:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0c52c1552e1cb...	Active	No	Create Route
10.1.0.0/16	pcx-04da84596e4ff0...	Active	No	Create Route
10.2.0.0/16	local	Active	No	Create Route Table



yakubu abdullahi yusuf

NextWork Student

nextwork.org

In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will use EC2 Instance Connect to connect to my first EC2 instance.

Step 6 - Connect to EC2 Instance 1

In this step, I will use EC2 Instance Connect to connect to instance 1.

Step 7 - Test VPC Peering

In this step, I will be sending test messages to instance 2, If I come across any error, I will solve it before proceeding.



Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to connect to my instance.

I was stopped from using EC2 Instance Connect as the EC2 instance requires a public IP.

No public IPv4 or IPv6 address assigned
With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect. Alternatively, you can try connecting using [EC2 Instance Connect Endpoint](#).

Instance ID
i-0889566337128e64b (Instance - NextWork VPC 1)

Connection type

☒ **Connect using a Public IP**
Connect using a public IPv4 or IPv6 address

☐ **Connect using a Private IP**
Connect using a private IP address and a VPC endpoint

☐ **Public IPv4 address**
-

☐ **IPv6 address**
-

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.



Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses provides fixed public IPs for EC2 instances so they remain reachable over the internet.

Associating an Elastic IP address resolved the error because it gave my EC2 instance a stable public IP address, allowing AWS services like EC2 Instance Connect.

Allocate Elastic IP address Info

Elastic IP address settings Info

Public IPv4 address pool

- ☒ Amazon's pool of IPv4 addresses
- ☐ Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) [Learn more](#)
- ☐ Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer owned pools found) [Learn more](#)
- ☐ Allocate using an IPv4 IPAM pool (option disabled because no public IPv4 IPAM pools with AWS service as EC2 were found)

Global static IP addresses

AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

[Create accelerator](#)

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.



Troubleshooting ping issues

To test VPC peering, I ran the command ping.

A successful ping test would validate my VPC peering connection because it confirms that the network connectivity and routing between the peered VPCs is correctly configured, which means instances in one VPC can reach instances in the other using their private IP addresses.

I had to update my second EC2 instance's security group because the default security rules were blocking inbound traffic from the peered VPC, which prevented communication between the two instances. I added a new rule that allows inbound ICMP.

```
[ec2-user@ip-10-1-9-165 ~]$ ping 10.2.4.0
PING 10.2.4.0 (10.2.4.0) 56(84) bytes of data:
64 bytes from 10.2.4.0: icmp_seq=314 ttl=127 time=0.420 ms
64 bytes from 10.2.4.0: icmp_seq=315 ttl=127 time=0.360 ms
64 bytes from 10.2.4.0: icmp_seq=316 ttl=127 time=0.485 ms
64 bytes from 10.2.4.0: icmp_seq=317 ttl=127 time=0.466 ms
64 bytes from 10.2.4.0: icmp_seq=318 ttl=127 time=0.434 ms
64 bytes from 10.2.4.0: icmp_seq=319 ttl=127 time=0.463 ms
64 bytes from 10.2.4.0: icmp_seq=320 ttl=127 time=0.494 ms
64 bytes from 10.2.4.0: icmp_seq=321 ttl=127 time=0.342 ms
64 bytes from 10.2.4.0: icmp_seq=322 ttl=127 time=0.455 ms
64 bytes from 10.2.4.0: icmp_seq=323 ttl=127 time=0.469 ms
64 bytes from 10.2.4.0: icmp_seq=324 ttl=127 time=0.477 ms
64 bytes from 10.2.4.0: icmp_seq=325 ttl=127 time=0.431 ms
64 bytes from 10.2.4.0: icmp_seq=326 ttl=127 time=0.430 ms
64 bytes from 10.2.4.0: icmp_seq=327 ttl=127 time=0.413 ms
64 bytes from 10.2.4.0: icmp_seq=328 ttl=127 time=0.451 ms
64 bytes from 10.2.4.0: icmp_seq=329 ttl=127 time=0.444 ms
64 bytes from 10.2.4.0: icmp_seq=330 ttl=127 time=0.438 ms
64 bytes from 10.2.4.0: icmp_seq=331 ttl=127 time=0.436 ms
64 bytes from 10.2.4.0: icmp_seq=332 ttl=127 time=0.481 ms
64 bytes from 10.2.4.0: icmp_seq=333 ttl=127 time=0.506 ms
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

